

New Customer Setup Runbook

SecureOps Master Template — fork-to-live tenant onboarding

New Customer Setup Runbook — SecureOps Master Template

Goal: stand up a brand-new customer on their own copy of the system — same features, zero WCSG data — and wire every integration back up, step by step.

The model (why this works): one shared mobile app serves every customer. Each customer gets their **own full backend** — their own copy of the code, their own database, their own domain, their own branding. Customers never share a server or a database. The phone app finds the right backend using a short **org code** (like `wcsg` or `rgp`) that this master project's directory resolves.

This project is the master. It plays three roles at once:

1. The golden template you fork for each new customer
2. WCSG's own live backend
3. The **org directory** (the phone app asks it "where does code `acme` live?") and the **only** project that pushes phone-app updates

Keep it healthy and don't fork **from** a customer copy — always fork from here.

What copies on a fork — and what doesn't

About "no real data": production data lives in the production database, which is created empty for each new deployment — so the customer copy starts clean automatically. The only thing to double-check is the **development** database inside the fork (Phase 2, step 4).

Phase 1 — Fork the template (2 min)

1. Open this project on Replit -> **Fork / Remix**.
2. Name it clearly: `SecureOps - <Customer Name>`.
3. Never do customer-specific feature work in the master; never fork from a customer copy.

Phase 2 — Clean the copy (10 min, inside the fork)

These committed settings carry over from WCSG and **must be changed** in the fork (App Settings -> Environment / Secrets panes, or ask the Agent in that fork to do it):

1. **Production env** — delete these two (they point at WCSG's domain; the new deploy then self-resolves to its own `.replit.app` address until you add a custom domain in Phase 6):
 - `APP_BASE_URL`
 - `ALLOWED_ORIGINS`
2. **Shared env** — change:
 - `DEMO_ADMIN_EMAIL` -> the **customer's** admin email (this becomes their master login)
 - `SUPER_ADMIN_EMAILS` -> same email; optionally add your own (comma-separated) to keep support access to their branding screen

- `ORG_CODE` -> their short code, e.g. `acme` (lowercase, no spaces — this is what staff type into the app)
- `ORG_DIRECTORY` -> **delete** in the fork (only the master keeps the directory)
- 3. **Seeding must stay ON for the first boot** (`SEED_DEMO_USERS` unset or `true`). This is what creates the customer's first admin account — if it's off on an empty database, nobody can ever log in.
- 4. **Check the fork's dev database** — if any WCSG data came across, wipe it and re-push the schema (`pnpm --filter @workspace/db run push`). Easiest: ask the Agent in the fork to "wipe the dev database and re-push the schema."
- 5. **NEVER add an `EXPO\_TOKEN` secret to a customer fork.** That token makes a project push phone-app updates on every publish. Only the master may have it — a customer fork with that token would push *its* code to **every customer's phones**.

Phase 3 — Secrets & integrations checklist (15–30 min, inside the fork)

Secrets never copy on a fork. Work through this table top to bottom:

Optional per-customer tuning (env, not secrets): `PAYROLL_TIMEZONE` (default America/Chicago), `EMERGENCY_CALL_NUMBER` (default 911), `GEOFENCE_RADIUS_MILES` (default 0.25).

Phase 4 — First publish (10 min, inside the fork)

1. Publish as **Reserved VM (always-on)** — **not** Autoscale (the radio and live features need an always-on server). A **fresh, empty production database** is created here.
2. After it's live, open `https://<their-address>/api/brand` in a browser — you should see text (JSON), not an error. If it errors persistently, the database schema didn't reach production: run `pnpm --filter @workspace/db run push` against the production database in the fork, or ask the fork's Agent to fix it.
3. Log in at `https://<their-address>/admin-portal/` with the `DEMO_ADMIN_EMAIL` + `DEMO_ADMIN_PASSWORD` you set. If an amber "degraded configuration" banner shows, it lists exactly which setting is still missing.

Phase 5 — Brand it (5 min, in their admin portal)

Log in as the admin -> **Platform** -> **Branding**. Set company name, short name, app name, tagline, the three brand colors, logo, and contact emails. Changes apply **live** — no republish needed. (The login screen intentionally stays "SecureOps Command" for everyone; the customer's brand appears everywhere after sign-in.)

Phase 6 — Custom domain (optional, 15 min + DNS wait)

1. Buy/connect the domain in the fork's Deployment -> Settings -> Domains.
2. Once it's serving, set in the fork's **production** env: `APP_BASE_URL=https://theirdomain.com` and `ALLOWED_ORIGINS=https://theirdomain.com`, then **republish**.
3. Email links and browser security now use their domain.

Phase 7 — Register the org code (5 min, back in THIS master project)

1. Edit the master's `ORG_DIRECTORY` env — add one line to the JSON list:


```
```json
{ "code": "acme", "name": "Acme Security", "apiBaseUrl": "https://theirdomain.com" }
```

`apiBaseUrl` is the origin only — no `/api`, no trailing slash. Use the `.replit.app` address if there's no custom domain yet (update it later when the domain lands).

2. **Republish the master** — the directory is read once at startup.
3. Verify: <https://secureops-command.replit.app/api/org-directory/resolve?code=acme> returns their address.
4. Their staff open the same SecureOps app from the store -> Connect screen -> type `acme` -> log in. The customer's own **invite QR / connect link** surface also works now (powered by the `ORG_CODE` you set in Phase 2).

## Phase 8 — Register in the fleet console (5 min)

In the control plane, add the customer: name, org code, backend address, and the `mgmtSecret` = the exact `CONTROL_PLANE_SHARED_SECRET` value you generated in Phase 3. Until both sides hold the same secret, the customer backend simply ignores control-plane requests (safe default).

Every new registration starts with a **Trial** lifecycle badge — that's expected even for a customer who's already committed to buying. Leave it as Trial until they've actually signed and started paying, then flip it in Phase 11.

## Phase 9 — Acceptance checklist (10 min)

Run through every line before handover:

- ☐ <https://<addr>/api/brand> loads (API + database healthy)
- ☐ Admin can log in; **no amber banner** in the portal
- ☐ Branding shows the customer's name/colors/logo after sign-in
- ☐ Send a staff **invitation** — the email arrives, from the right address
- ☐ Public **apply** page loads: <https://<addr>/admin-portal/apply>
- ☐ Phone: same app -> Connect -> their org code -> login works
- ☐ Radio: hold Talk on one device, hear it clearly on another
- ☐ Upload a document (e.g. a license) — it saves and re-opens
- ☐ Org code resolves from the master directory (Phase 7, step 3)
- ☐ Customer appears healthy in the control plane
- ☐ Run the config preflight from a Shell **inside the fork**:  
`pnpm --filter @workspace/scripts run check-tenant-config`.

It checks the fork's own production environment (database, session secret, org identity, object storage, email, Twilio, control-plane secret, brand env defaults) and prints every gap at once — fix any line marked **FAIL** before telling the customer they're live; lines marked **WARN** are worth a look but don't block handover.

## Phase 10 — Handover & lockdown (10 min)

1. Customer admin logs in and **changes their password**.
2. The demo staff accounts (`officer@` / `lead@` / `guest@secureops.com`) are seeded for testing. Once the customer is live, set `SEED_DEMO_USERS=false` in production env and republish, then deactivate those demo users in the Users grid.
3. Confirm who holds super-admin (`SUPER_ADMIN_EMAILS`) — that's who can change branding and platform settings.
4. Send them the legal docs (`legal/` folder: Master Subscription Agreement + User Agreement).

## Phase 11 — Flip Trial -> Paid (once they sign)

The fleet console tracks each customer's billing lifecycle (Trial or Paid) —

a manual status flag, not in-app billing. New registrations always start in **Trial** (Phase 8). Once the customer signs and starts paying:

- 1. Open the control plane dashboard -> find the customer's row.
- 2. Click **Mark Paid** (shown only while a customer is in Trial), or open **Edit** and switch the Lifecycle field to Paid.
- 3. This stamps a conversion timestamp and switches their badge from amber "Trial" to green "Paid" in both the fleet list and their detail view.  
Nothing about the customer's own backend changes — this is purely a record in the operator's registry.

### Quick-reference: the whole flow

Fork master -> clean env (Phase 2) -> re-enter secrets & reconnect integrations (Phase 3) -> publish Reserved VM, seeding ON (Phase 4) -> brand in-app (Phase 5) -> domain (Phase 6) -> add org code to master directory + republish master (Phase 7) -> control plane (Phase 8) -> verify (Phase 9) -> handover (Phase 10).

#### Golden rules

- Always fork from the master, never from a customer copy.
- Seeding ON for every first boot — off too early means permanent lockout.
- **EXPO\_TOKEN** lives **only** in the master.
- One customer = one backend = one database = one bucket. Nothing shared.
- The master's **ORG\_DIRECTORY** is the single source of truth for org codes; every change needs a master republish.

	Copies automatically	You must redo it	
All code & features	'		
Committed settings (`.replit` env blocks)	' *(some must be changed — Phase 2)*		
Secrets (passwords, API keys)		'L re-enter (Phase 3)	
Replit integrations/connectors (Twilio, Gemini AI)		'L reconnect (Phase 3)	
File storage bucket (uploaded documents, logos)		'L create fresh (Phase 3)	
Production database		' *created fresh & empty on first publish — real WCSG data never transfers*	
Custom domain, deployment		'L set up new (Phases 4 & 6)	
Secret / integration	What it powers	Required?	Where the va
---	---	---	---
`SESSION_SECRET`	Login security	' (deploy fails without it)	Generate a f characters — never reuse

`DEMO_ADMIN_PASSWORD`	Customer's first admin login	'	Choose a strong password and change it after deployment
`SMTP_HOST` / `SMTP_PORT` / `SMTP_USER` / `SMTP_PASS` / `SMTP_FROM`	All outbound email (invites, password resets, onboarding links)	' for HR flows	Ideally a mail domain (their name); account works with `RESEND_API_KEY` and `RESEND_EMAIL_PROVIDER`
`LIVEKIT_URL` / `LIVEKIT_API_KEY` / `LIVEKIT_API_SECRET`	Push-to-talk radio	' if they use radio	Create a new project per customer — keeps each customer's data and billing separate
`CONTROL_PLANE_SHARED_SECRET`	Lets your fleet console manage this customer	Recommended	Generate a fresh secret and paste the same one in Phase 2
Twilio (Replit integration pane)	SMS alerts (emergency, shift notices)	Optional	Reconnect in the integration pane; without this, no work
Gemini AI (Replit integration pane)	AI features	Only if used	Reconnect in the integration pane
`SCHEDULER_BASE_URL` / `SCHEDULER_SHARED_SECRET`	External scheduler sync	Only if used	From the customer's scheduler
App Storage bucket	Uploaded documents, licenses, logos	'	Create a new bucket (Object Storage Agent). Files are stored here









